



# Aurora Report

A hands-on tutorial for Playwright test reports

**Version** 1.1.0

**Updated** 21 September 2026

**Source** [github.com/chinnu-82/chinnu\\_report](https://github.com/chinnu-82/chinnu_report)

# Contents

---

## 1. Part 0

The big picture

---

## 2. Part 1

Run the demo

---

## 3. Part 2

Tour of the report

---

## 4. Part 3

Investigate your first failure

---

## 5. Part 4

The flaky test

---

## 6. Part 5

A passing test that hides errors

---

## 7. Part 6

Watch the test instead of reading it

---

## 8. Part 7

The timeline

---

## 9. Part 8

Make your own test tell a story

---

## 10. Part 9

Use it on your own project

---

## 11. Glossary

---

## 12. What next?

---

# Aurora Report Tutorial

A hands-on tour for people who are new to Aurora Report — and new to test reports in general. You'll run a demo test suite that fails on purpose, then use the report to work out *why* it failed.

**Time:** about 30 minutes · **You need:** [Node.js](#) 18+ and this repository.

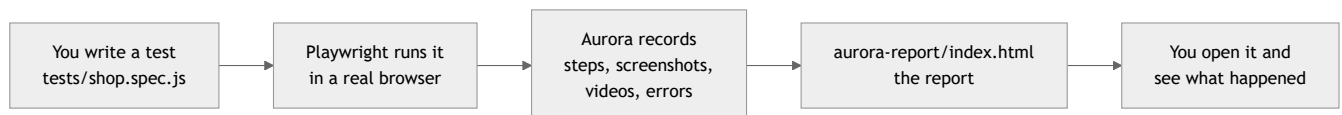
 **Prefer to print or read offline?** [Download the PDF](#) — the same tutorial, 30 pages, with the answers to the exercises shown.  
Looking for the full reference instead? See [GUIDE.md](#).

## Part 0 — The big picture

Three tools with three jobs:

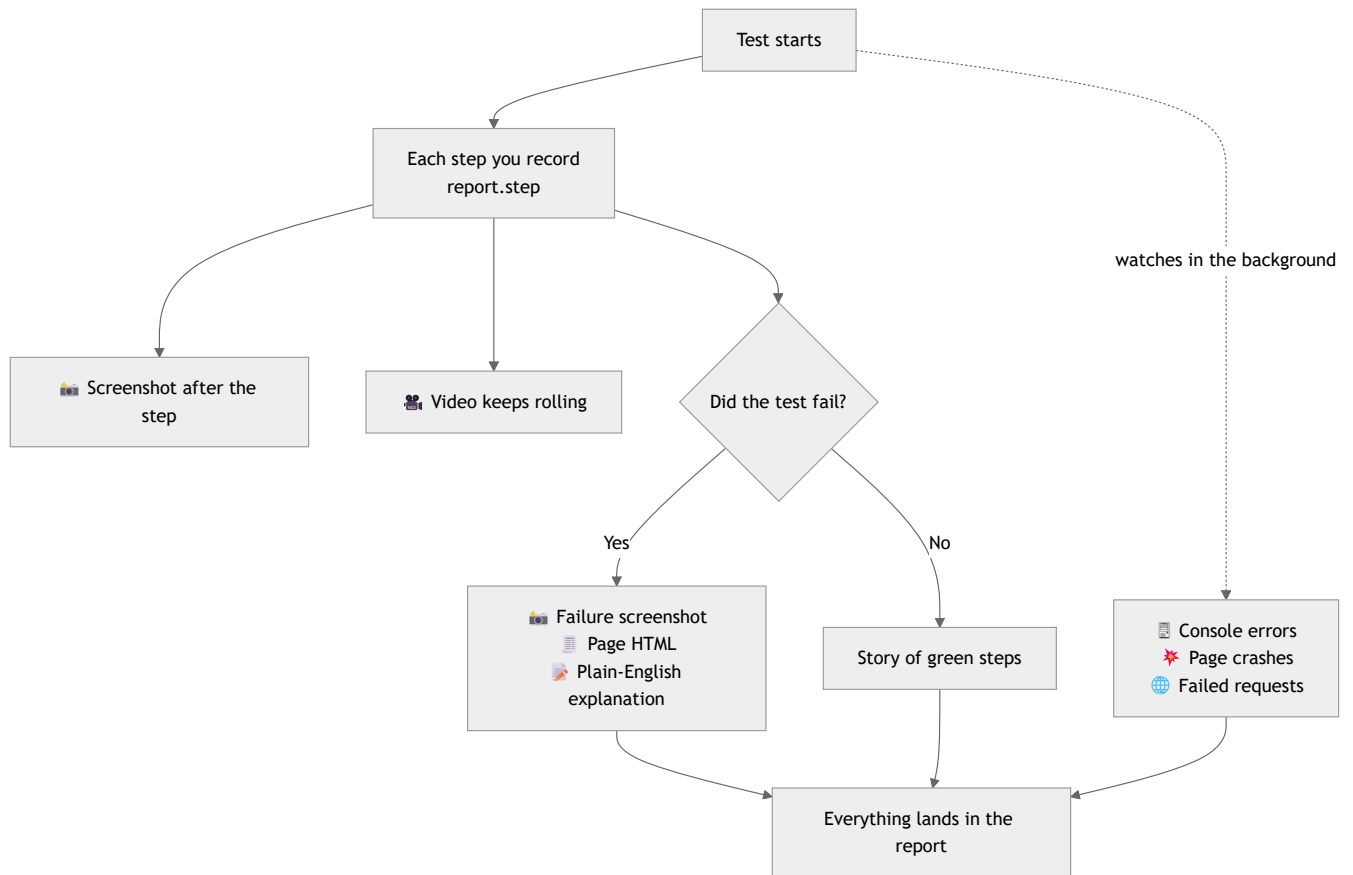
| Tool                 | Job                                                      |
|----------------------|----------------------------------------------------------|
| <b>Playwright</b>    | Opens a real browser and runs your tests                 |
| <b>Aurora Report</b> | Watches that run and records what happened               |
| <b>The report</b>    | An HTML page you open in your browser to see the results |

Here's the whole flow:



**Why not just read the terminal?** The terminal tells you *a test failed*. The report shows you *what the page looked like when it failed, what the test did just before, and whether this is new*.

## What Aurora records during a single test



The parts marked "*watches in the background*" need no code at all. You get them just by using Aurora.

---

## Part 1 — Run the demo

This repository contains a small pretend shop ( `demo-app/` ) and tests for it ( `tests/shop.spec.js` ). Some tests fail **on purpose**, so the report has something interesting to show.

### The easy way (Windows)

Double-click `start-demo.bat` . It installs what's needed, runs the tests, and opens the report.

### The manual way (any OS)

```
npm install
npx playwright install chromium
npx playwright test
```

You'll see this at the end:

```
4 failed
2 flaky
2 skipped
10 passed (13.9s)

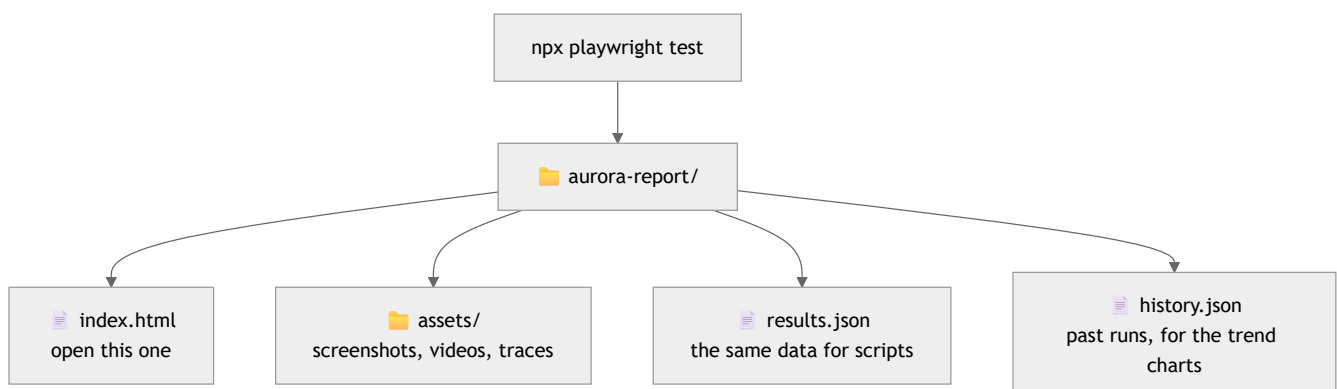
🌟 Aurora report 10 passed · 4 failed · 2 flaky · 2 skipped
file:///C:/.../aurora-report/index.html
```

**"My tests failed!"** That's expected here. The demo has deliberate bugs so you have something to investigate. On CI, `npx playwright test` exiting with code 1 means *some tests failed*.

Open the report:

```
npx aurora-report open
```

### What just got created

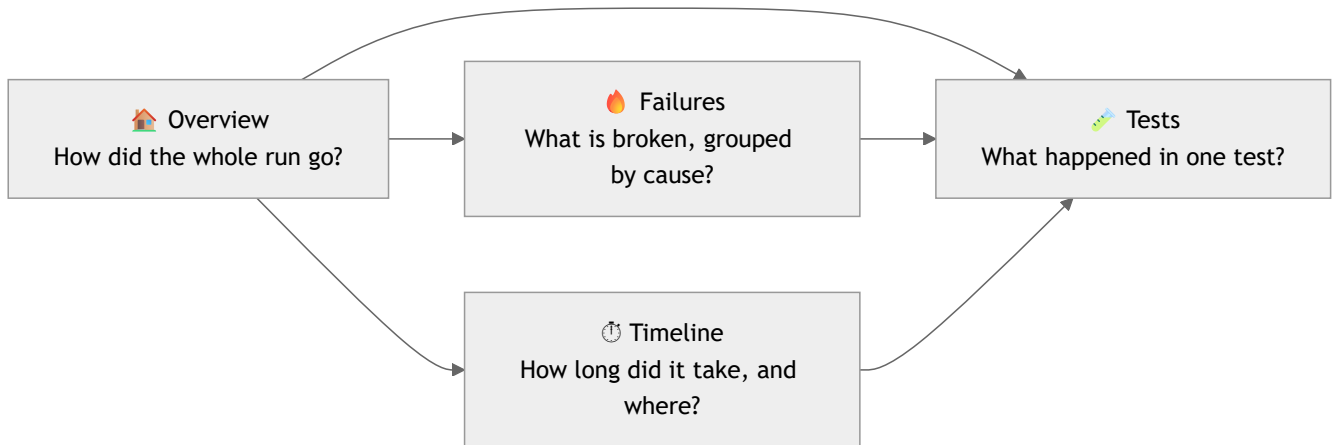


Only `index.html` matters for this tutorial. Keep the whole folder together, because the page loads screenshots from `assets/`.

---

## Part 2 — Tour of the report

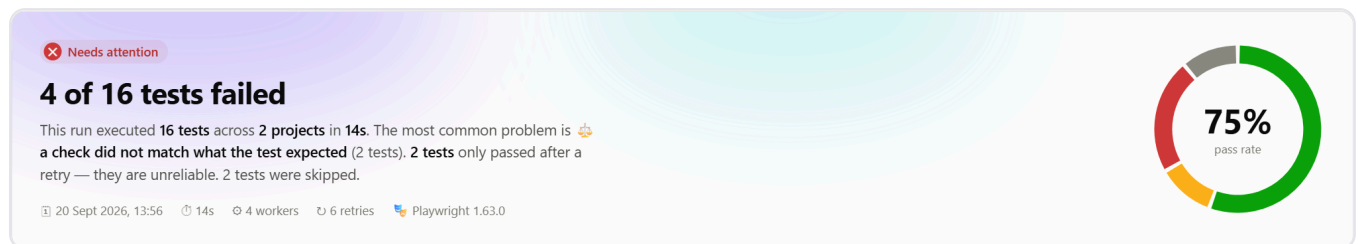
The report has four pages. This is the map:



Press **1**, **2**, **3**, **4** to switch pages.

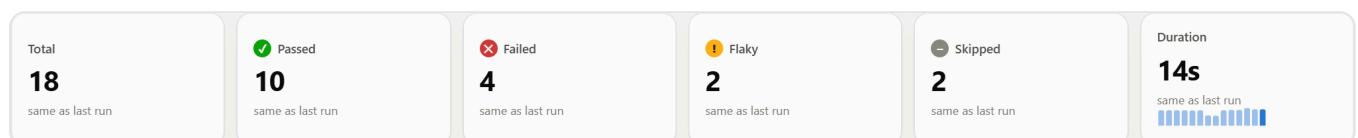
### The headline

The top of the Overview says what happened in plain English — no chart reading required.



Read it out loud: *4 of 16 tests failed... the most common problem is a check did not match what the test expected... 2 tests only passed after a retry.* That's the entire run in three sentences. The ring on the right is the pass rate.

### The number tiles



Each tile compares this run with the previous one, so ▲ or ▼ tells you whether things got better or worse.

**Click a tile** to jump to those tests.

- **Failed** — the test did not pass, even after retries.
- **Flaky** — it failed, then passed when retried. Nothing is broken for the user, but the test can't be trusted.
- **Skipped** — the test was deliberately not run.

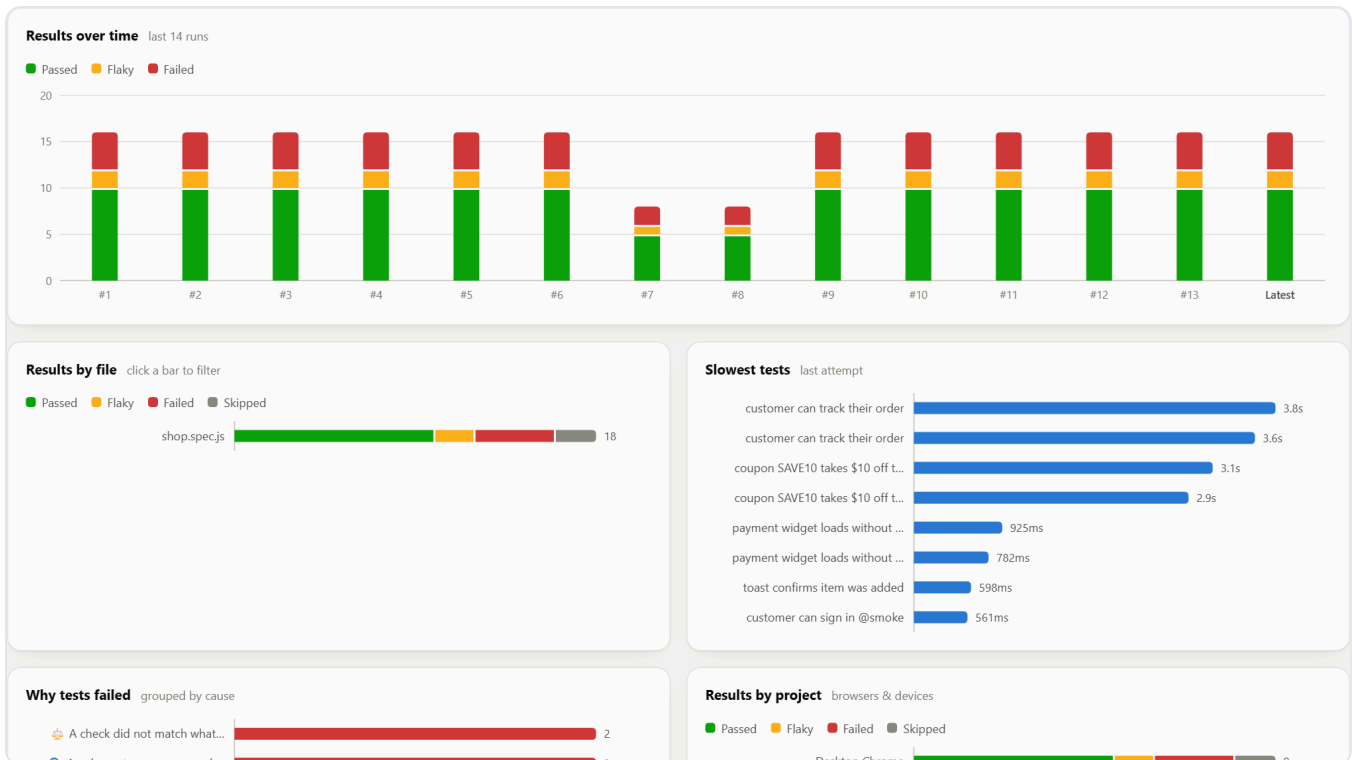
## What needs your attention

 **A check did not match what the test expected**  
The test expected "\$75" but the app gave "\$85". 2 tests

 **An element never appeared on the page**  
The test waited 3s for getByRole('button', { name: 'Track order' }) but it never showed up. 2 tests

Aurora groups failures by **cause**, not by test. If 12 tests fail because one button was renamed, you see one row, not twelve. Start at the top.

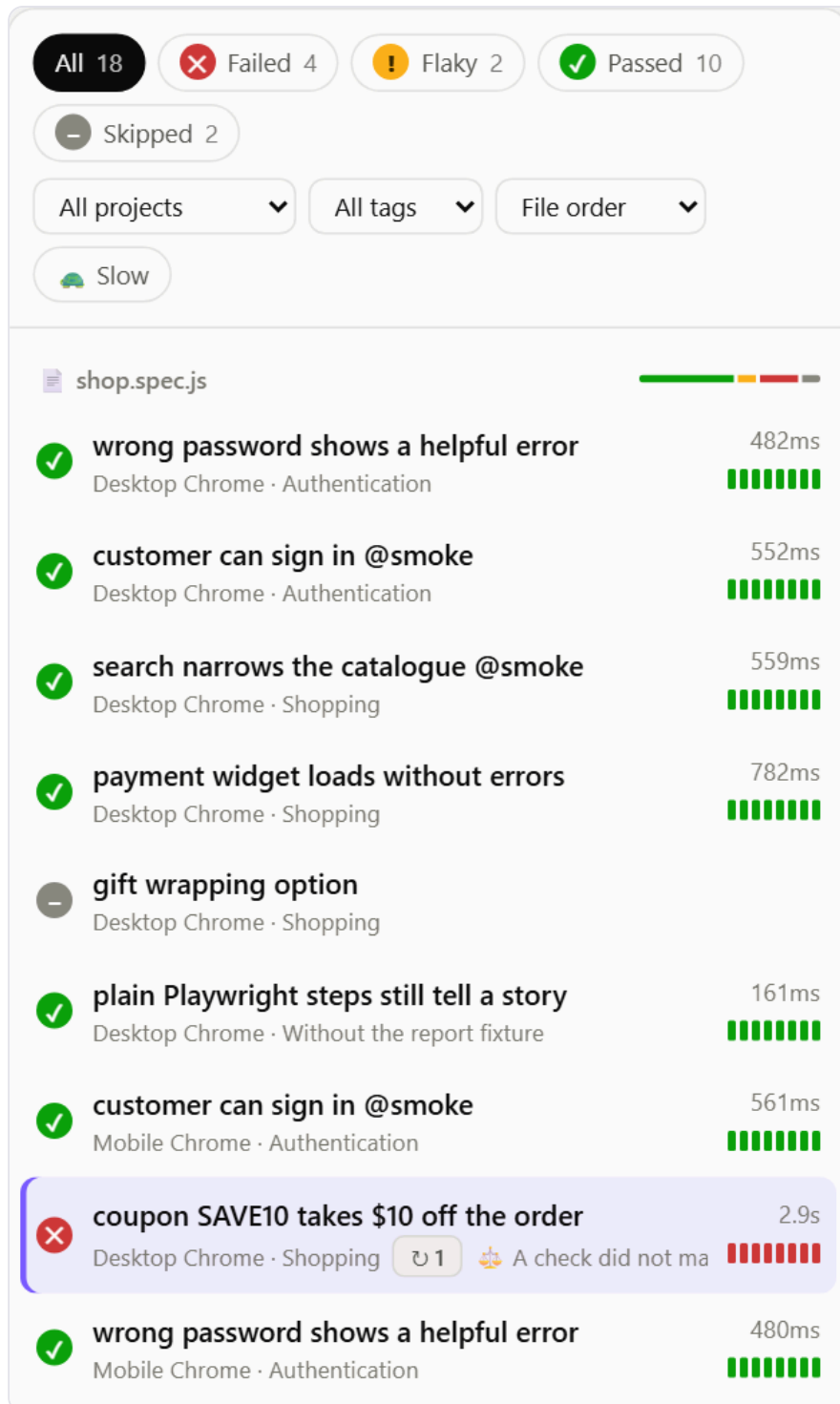
## The charts



- **Results over time** — each bar is one run. Growing red means things are getting worse.
- **Results by file** — which spec file is unhealthy.
- **Slowest tests** — where your run time goes.
- **Why tests failed** — failure causes, largest first.

## Finding one test

Press **2** for the Tests page. The list on the left is how you find anything:



- **Status chips** — show only failed, flaky, passed or skipped tests.
- **Dropdowns** — filter by project (browser), by tag such as `@smoke`, or re-sort.
- **The little coloured bars** on the right of each row are that test's recent runs: green passed, red failed, amber flaky. A row of green with one red at the end means it just broke.

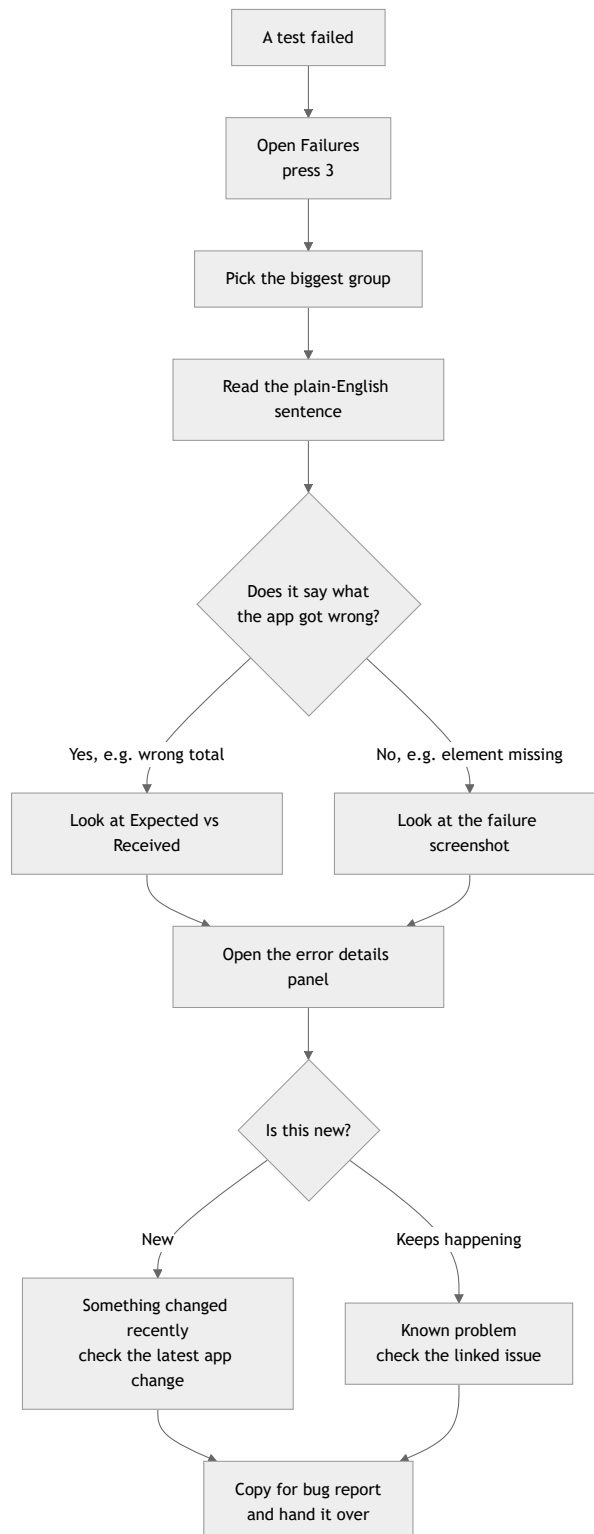
**Exercise 1.** Look at your own report. How many tests ran, and what is the single most common failure cause?

✓ **Answer for the demo**

16 tests ran (18 including skipped). The most common cause is "A check did not match what the test expected" — 2 tests.

## Part 3 — Investigate your first failure

Here's the routine to follow whenever something is red:



Let's do it with the demo.

### Step 1 — Open the Failures page

Press **3**. The first group looks like this:


**A check did not match what the test expected**  
 The test expected "\$75" but the app gave "\$85".
 2 tests

Error: expect(locator).toHaveText(expected) failed
 Details >


**coupon SAVE10 takes \$10 off the order**  
 Desktop Chrome · shop.spec.js
 Error details

2.9s


**coupon SAVE10 takes \$10 off the order**  
 Mobile Chrome · shop.spec.js
 Error details

3.1s

It tells you:

- **What went wrong:** the test expected \$75 but the app gave \$85
- **How many tests:** 2 — the same test on two browsers
- Two ways in: **Details >** for the cause, or a test name to open that test

## Step 2 — Open the test

Click **coupon SAVE10 takes \$10 off the order · Desktop Chrome**.

Desktop Chrome
 shop.spec.js:61 > Shopping

**coupon SAVE10 takes \$10 off the order**

 **Failed**
2.9s
high
Checkout
SHOP-142
3 browser errors

 First run 3.0s
  **Retry 1** 2.9s

The header shows the status, how long it took, the severity and owner you set in the test, the linked issue, and the retry tabs — **First run** and **Retry 1**, because Playwright retried it once.

## Step 3 — Read "Why it failed"


**WHY IT FAILED**  
**A check did not match what the test expected**  
 The test expected "\$75" but the app gave "\$85".

✓ EXPECTED  
 \$75

✗ RECEIVED  
 \$85

Failed at step **Total shows the discount**

 getByTestId('total')
 
 shop.spec.js:85

Compare Expected and Received — the highlighted part is exactly what differs.

 **See full details**
 where it happened, what Playwright tried, browser clues, history & fixes

This is the important part, and it's four lines:

1. **The headline** — *A check did not match what the test expected.*
2. **Expected vs Received** — \$75 versus \$85, with the difference highlighted. The coupon wasn't applied.

3. **Where** — it failed at the step *"Total shows the discount"*, on the element `getByTestId('total')`, at `shop.spec.js:85`.
4. **One tip** on what to do next.

You already know enough to file a bug: **the SAVE10 coupon does not reduce the total.**

## Step 4 — Get the full picture

Click  **See full details** (or click any error anywhere in the report). A panel slides in:

**ERROR DETAILS · COUPON SAVE10 TAKES \$10 OFF THE ORDER · RETRY 1**×

**A check did not match what the test expected**  
The test expected "\$75" but the app gave "\$85".

 Copy for bug report  Copy link

**1 What was expected vs. what happened**

✓ EXPECTED  
\$75

✗ RECEIVED  
\$85

The highlighted part is the difference.

**2 Where it happened**

Step**Total shows the discount** · started at +557ms, failed after 2.0s

Element `getByTestId('total')`  Copy

Waited 2.0s

Code `shop.spec.js:85`

tests/shop.spec.js:85

```
83     });
84     await report.step('Total shows the discount', async () => {
85       await expect(page.getByTestId('total')).toHaveText('$75', { timeout: 2000 });
86     }, { highlight: page.getByTestId('total') });
87   });
88 }
```

**3 What the page looked like**

Nimbus Store ada@nimbus.test Cart (2)

Checkout

Cloud Hoodie

\$60

Storm Mug

\$25

It answers the questions you'd ask next, in order:



```
**Test:** coupon SAVE10 takes $10 off the order (Desktop Chrome)
**File:** tests/shop.spec.js:61
**Problem:** A check did not match what the test expected. The test expected "$75" but the app
gave "$85".
**Failed step:** Total shows the discount
**Expected:** $75
**Received:** $85
**Element:** `getByTestId('total')`
```

Use  **Copy link** to send a teammate a link that opens the report on this exact error.

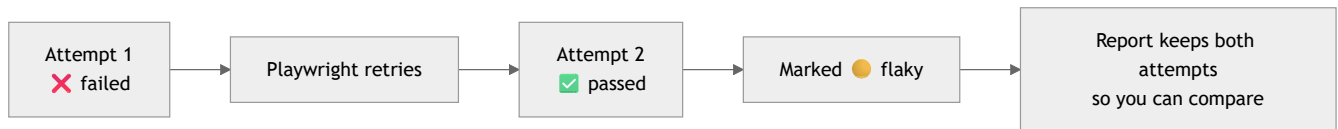
**Exercise 2.** Investigate the other failing test, *customer can track their order*. What's different about this failure, and what does the screenshot tell you?

✓ **Answer**

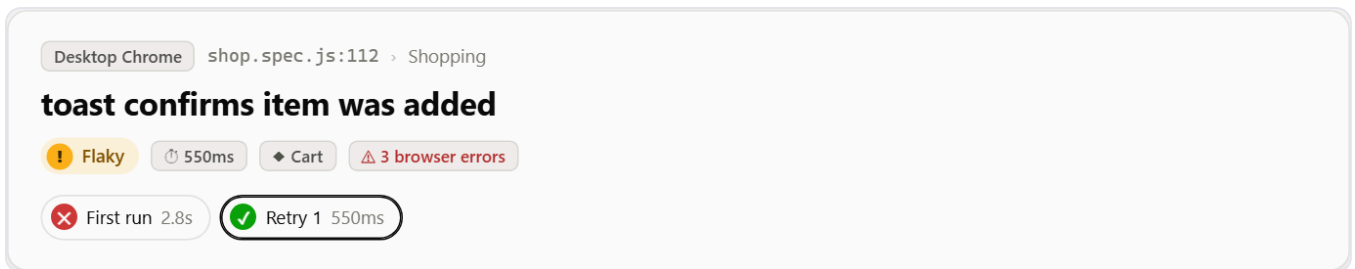
It's a missing element, not a wrong value: "*The test waited 3s for getByRole('button', { name: 'Track order' }) but it never showed up.*" The failure screenshot shows the shop page with no Track order button anywhere — the feature doesn't exist yet, so the test, not the app, is wrong.

## Part 4 — The flaky test

A flaky test is one that fails, then passes when run again. Nothing is broken for the user — but you can't trust the test.

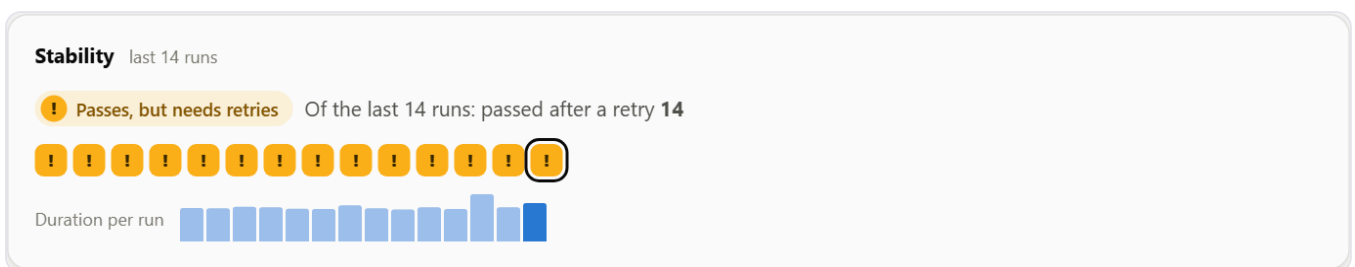


Open **toast confirms item was added**. The header has two tabs:



Click **First run** to see the failure and its screenshot; click **Retry 1** to see the passing version. Comparing the two is usually enough to spot a timing problem.

Scroll to **Stability**:



*Passes, but needs retries — of the last 14 runs: passed after a retry 14.* That's a test with a permanent timing problem, not a one-off hiccup. In the demo this is on purpose: the test waits too long before checking a message that disappears after 1.5 seconds.

**Exercise 3.** Open the **First run** tab and read the error. Why did it fail?

### ✓ Answer

"An element never appeared on the page — the test waited 500ms for `getByText('Added to cart')...`". The toast had already disappeared by the time the test looked for it. The fix is to check for the toast immediately, not after a wait.

## Part 5 — A passing test that hides errors

Green doesn't always mean healthy. Open **payment widget loads without errors** — it passed, but its header carries a red **4 browser errors** badge. Click it:

**Browser diagnostics** captured automatically — no code needed

 **Uncaught page errors** 1

| At     | Error                                          |
|--------|------------------------------------------------|
| +401ms | <b>PaymentWidget is not defined</b><br>▶ stack |

 **Console** 2

| At    | Level          | Message                                                                                                                                  |
|-------|----------------|------------------------------------------------------------------------------------------------------------------------------------------|
| +12ms | <b>✗ error</b> | <b>Failed to load resource: net::ERR_FILE_NOT_FOUND</b><br>file:///C:/Users/Vivek/projects/TestReport/demo-app/missing-hero-banner.png:0 |
| +13ms | <b>✗ error</b> | <b>Analytics script failed to load: blocked by client</b><br>file:///C:/Users/Vivek/projects/TestReport/demo-app/index.html:65           |

 **Failed network requests** 1

| At    | Status     | Request                                                                                                                   |
|-------|------------|---------------------------------------------------------------------------------------------------------------------------|
| +12ms | <b>ERR</b> | <b>GET file:///C:/Users/Vivek/projects/TestReport/demo-app/missing-hero-banner.png</b><br>net::ERR_FILE_NOT_FOUND · image |

Aurora recorded, with no code in the test:

- ✗ **Uncaught page errors** — `PaymentWidget is not defined`. A real JavaScript crash in the app that the test never checked for.
- 📄 **Console errors** — a failed analytics script.
- 🌐 **Failed network requests** — a missing image, `net::ERR_FILE_NOT_FOUND`.

The test only checked that a button could be clicked, so it passed. The report shows the crash anyway.

**Tip.** When someone says "but the tests are green", this panel is where you look.

## Part 6 — Watch the test instead of reading it

### The story

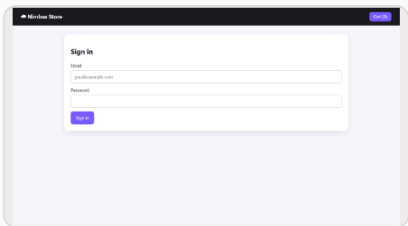
Every test has a **Test story**: each recorded step with a screenshot.

**Test story** 7 steps · 1 failed · 7 screenshots

[▶ Play story](#)

1

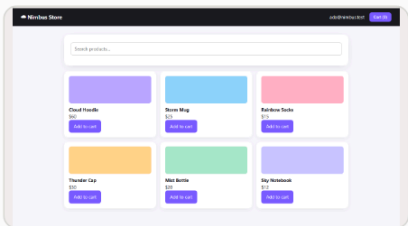
Open the store



37ms · +0ms

2

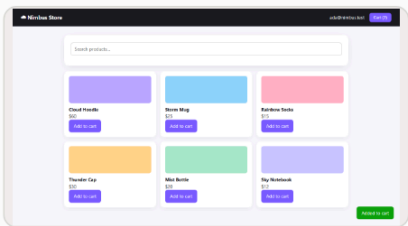
Sign in as a returning customer



105ms · +83ms

3

Add a hoodie and a mug to the cart



79ms · +213ms

4

Go to checkout

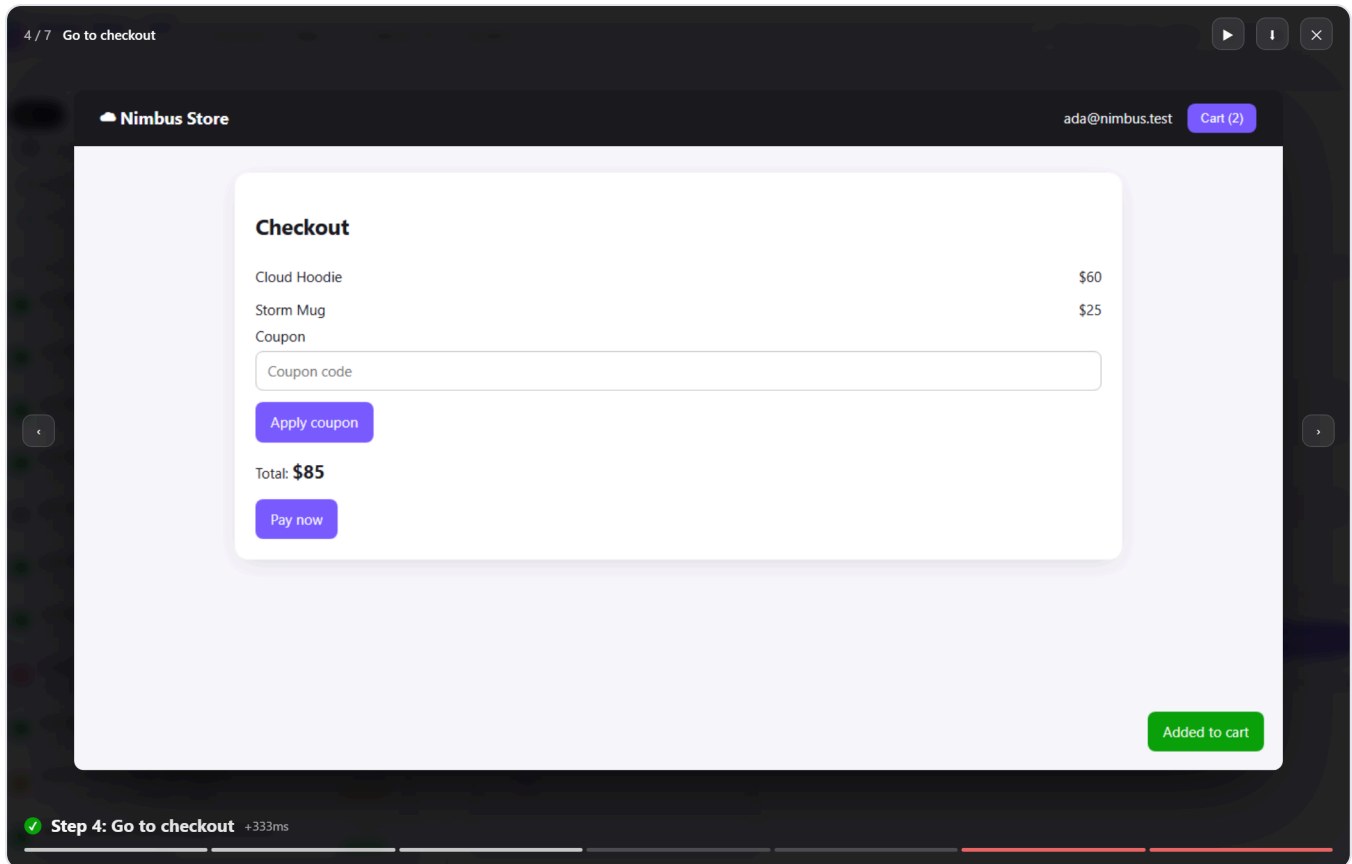


42ms · +333ms

Read down the page and you see the customer signing in, adding items, opening checkout, and the step that failed — in pictures.

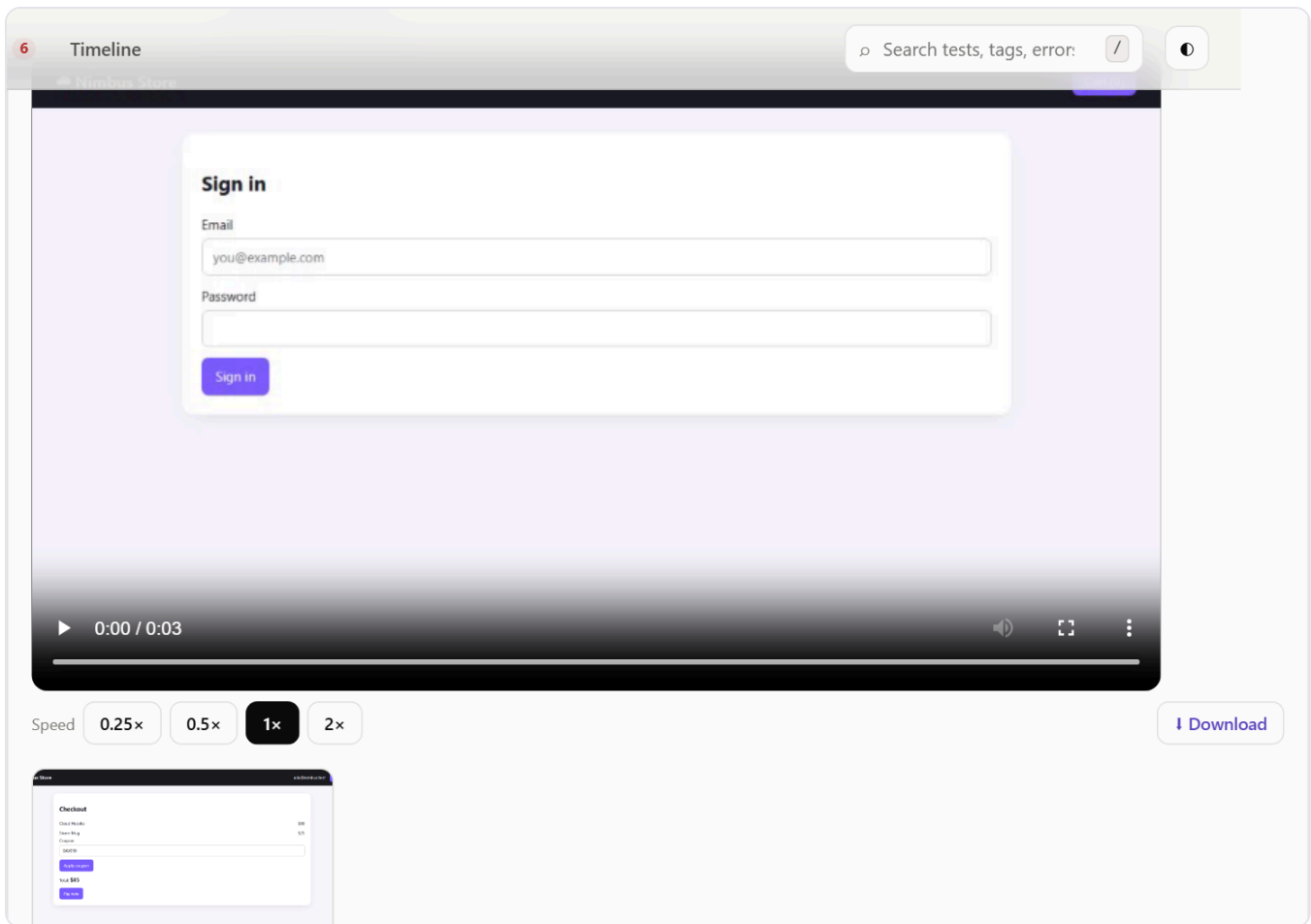
### Play story

Click ▶ **Play story** for a full-screen slideshow of the test:

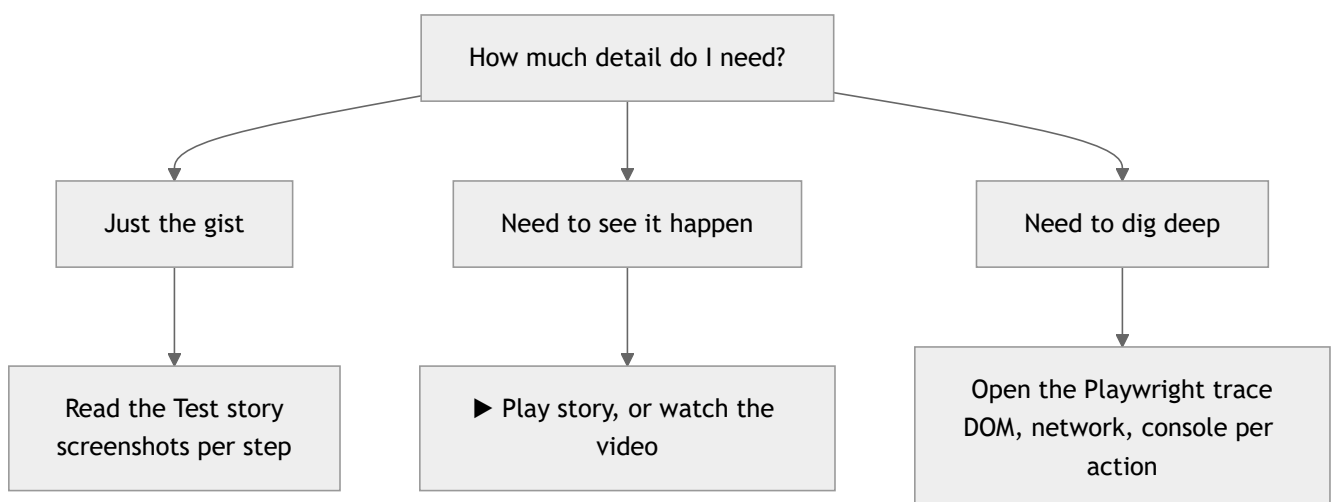


Use `←` `→` to move, `Space` to pause, `Esc` to close. Red marks on the progress bar are failed steps. This is the fastest way to show a non-technical colleague what the test does.

## Video, screenshots and trace

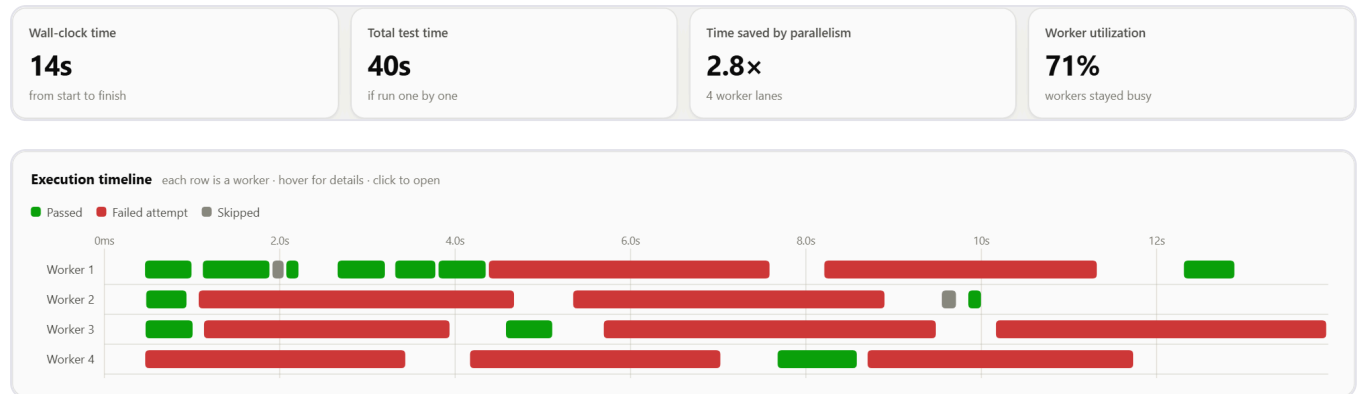


- **Video** of the run, with 0.25x–2x speed buttons. Slow it down to catch a flash of an error message.
- **Failure screenshot** and other screenshots.
- **Playwright trace** — copy the command to open Playwright's own step-by-step debugger with DOM snapshots and network details.



## Part 7 — The timeline

Press 4.



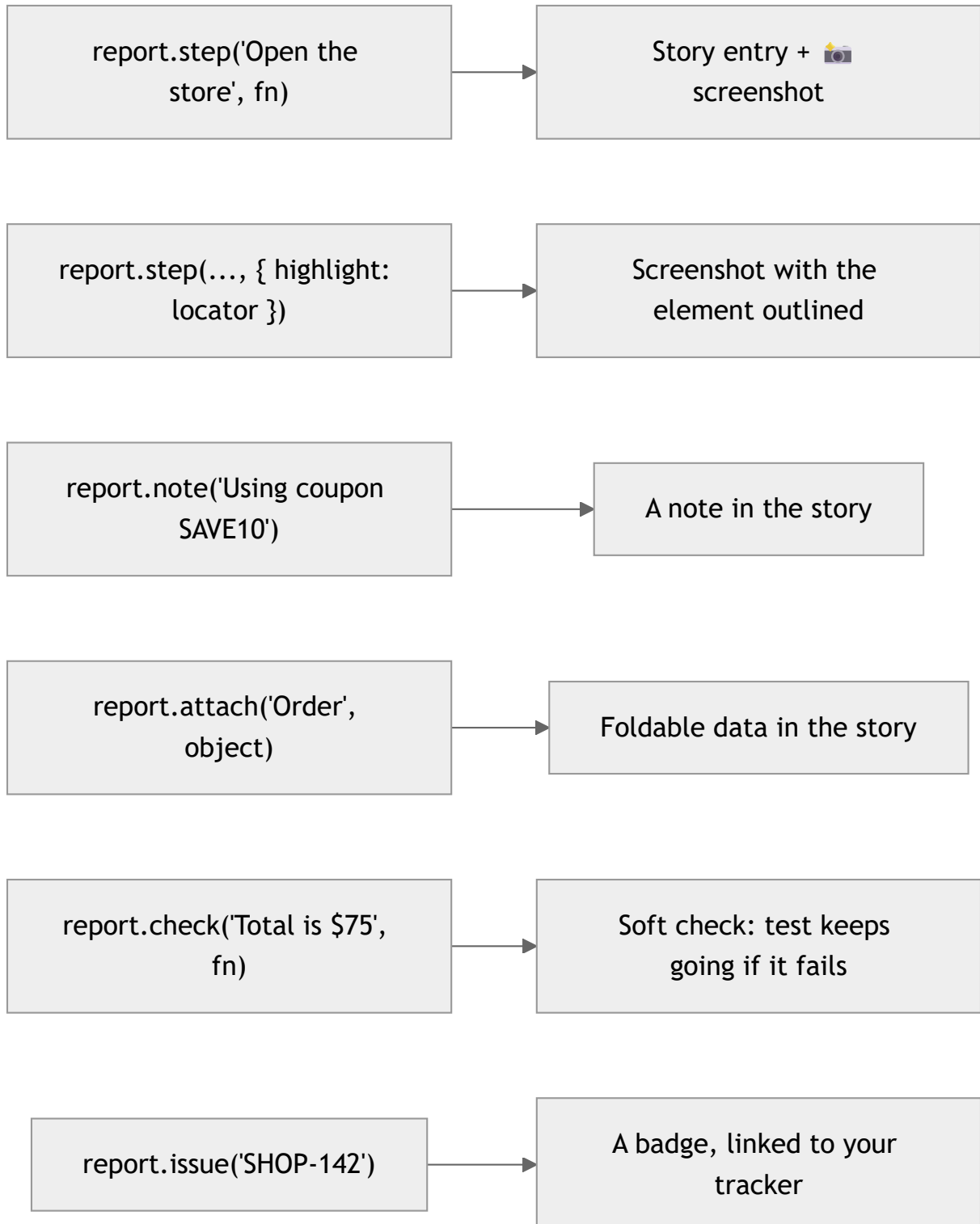
Each row is a worker — a parallel browser. Green is a passing test, red a failing attempt. Use this page to answer:

- **Why did the run take so long?** Look for one long bar holding everything up.
- **Am I using my machine well?** Low *worker utilization* means workers sat idle, often waiting on a shared resource.
- **Could I run more in parallel?** *Total test time* far above *wall-clock time* means parallelism is already helping.

## Part 8 — Make your own test tell a story

---

So far you've read a report. Now produce one. Each line of code maps to something you can see:



Create `tests/my-first.spec.js`:

```

const path = require('path');
const { pathToFileURL } = require('url');
const { test, expect } = require('..../src'); // in your own project: require('aurora-report')

const APP = pathToFileURL(path.join(__dirname, '..', 'demo-app', 'index.html')).href;

test('my first story', async ({ page, report }) => {
  report.owner('Me');
  report.feature('Learning');

  await report.step('Open the store', async () => {
    await page.goto(APP);
  });

  await report.step('Sign in', async () => {
    await page.getByLabel('Email').fill('learner@example.com');
    await page.getByLabel('Password').fill('secret');
    await page.getByRole('button', { name: 'Sign in' }).click();
  });

  report.note('The catalogue should now show 6 products');

  await report.step('Six products are listed', async () => {
    await expect(page.getByTestId('product')).toHaveCount(6);
  }, { highlight: page.locator('#products') }); // outlines the product grid

  await report.attach('What I checked', { products: 6, user: 'learner@example.com' });
});

```

Run just this test and open the report:

```

npx playwright test tests/my-first.spec.js --project "Desktop Chrome"
npx aurora-report open

```

In the report you should see your test with a three-step story, a screenshot per step, your note between steps 2 and 3, the product grid outlined in pink in the last screenshot, and your attached data at the bottom.

**Exercise 4.** Make your test fail on purpose — change `toHaveCount(6)` to `toHaveCount(7)` — and run it again. Which failure category does Aurora give it, and what does Expected vs Received say?

✓ Answer

"A check did not match what the test expected", with Expected 7 and Received 6. The failure screenshot shows the six products, and the story marks the third step red.

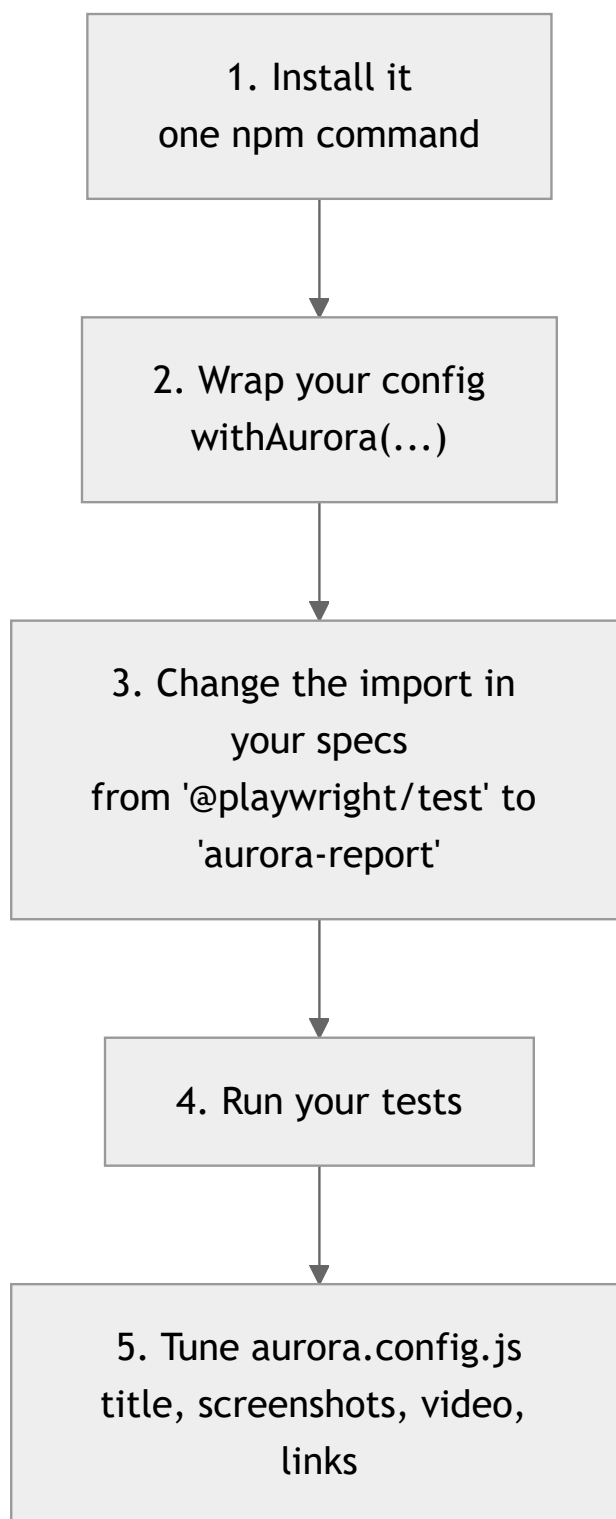
## Tips for stories worth reading

| Instead of                                       | Write                                                             |
|--------------------------------------------------|-------------------------------------------------------------------|
| <code>await report.step('click btn', ...)</code> | <code>await report.step('Customer submits the order', ...)</code> |
| One giant step for the whole test                | One step per thing the user does                                  |
| <code>report.step('check')</code>                | <code>report.step('Cart badge shows 2 items')</code>              |

Name each step after **what should be true afterwards**. Then the story reads like a description of the feature, and a failed step tells you what stopped being true.

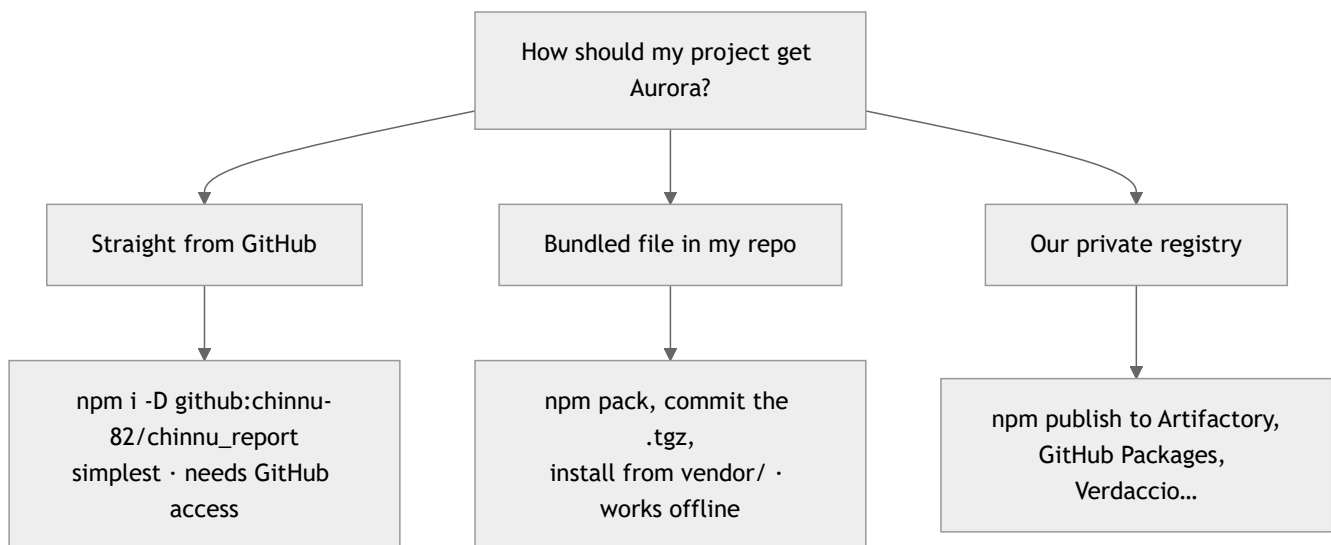
## Part 9 — Use it on your own project

---



### Step 1 — Install it

Aurora is **not on the public npm registry**, so `npm install aurora-report` on its own will not find it. Pick one of these three instead.



### a) Straight from GitHub — the simplest

```
npm install --save-dev @playwright/test github:chinnu-82/chinnu_report
```

It appears in `node_modules` as `aurora-report`. There is no build step. Pin it so upgrades are deliberate:

```
npm install --save-dev github:chinnu-82/chinnu_report#<commit-sha>
```

### b) A bundled file in your repo — no network needed

Use this when CI has no access to GitHub, or you want the exact bytes pinned. In a checkout of the report repo:

```
npm pack # produces aurora-report-1.1.0.tgz
```

Copy that file into your project (for example `vendor/`), commit it, and point `package.json` at it:

```
"devDependencies": {
  "@playwright/test": "^1.63.0",
  "aurora-report": "file:vendor/aurora-report-1.1.0.tgz"
}
```

Then `npm install`. Anyone who clones your repo gets the reporter with no extra steps — this is what the [Shopify store example](#) does.

### c) Your own npm registry

If your team runs Artifactory, GitHub Packages or Verdaccio, `npm publish` it there once and install it by name like any other package.

**You need** Node 18 or newer, and `@playwright/test` 1.40 or newer, which stays your own dependency.

## Step 2 — Wrap your Playwright config

```
// playwright.config.js
const { defineConfig } = require('@playwright/test');
const { withAurora } = require('aurora-report');

module.exports = withAurora(
  defineConfig({ testDir: './tests' }),
  { title: 'My App' },
);
```

Your existing reporters and `use` settings are kept.

## Step 3 — Change the import in your specs

```
- const { test, expect } = require('@playwright/test');
+ const { test, expect } = require('aurora-report');
```

Your tests appear in the report either way. The import change is what gives you the `report` fixture — steps, highlighted screenshots, soft checks — plus automatic capture of console errors, page crashes and failed requests.

## Step 4 — Run

```
npx playwright test
npx aurora-report open
```

## Step 5 — Tune it

Create `aurora.config.js` when you want to change the title, the accent colour, how much is captured, or where reports go:

```
const { defineAuroraConfig } = require('aurora-report');

module.exports = defineAuroraConfig({
  title: 'My App - E2E',
  theme: { accent: '#1f8a70' },
  timestampedRuns: true, // keep every run in its own dated folder
  capture: { stepScreenshots: 'on', video: 'retain-on-failure' },
});
```

Every step, with copy-paste examples for JavaScript, TypeScript and ES modules, is in [GUIDE.md](#):

- [Install](#) · [Connect it to Playwright](#) · [Update your tests](#)
- [Recording steps](#) · [Screenshots](#) · [Videos and traces](#)
- [Configuration](#) · [Running in CI](#) · [Troubleshooting](#)

## Keyboard shortcuts worth remembering

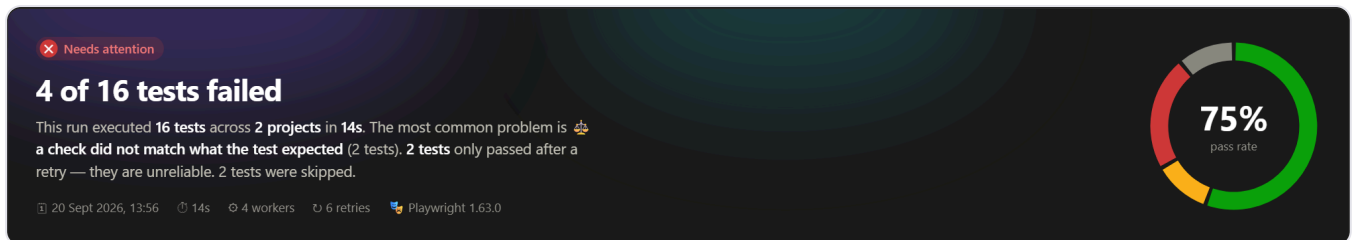
| Key                                                                     | Does                                                                                                   |
|-------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <a href="#">1</a> <a href="#">2</a> <a href="#">3</a> <a href="#">4</a> | <a href="#">Overview</a> · <a href="#">Tests</a> · <a href="#">Failures</a> · <a href="#">Timeline</a> |
| <a href="#">/</a>                                                       | <a href="#">Search</a>                                                                                 |
| <a href="#">j</a> / <a href="#">k</a>                                   | <a href="#">Next</a> / <a href="#">previous test</a>                                                   |
| <a href="#">Esc</a>                                                     | <a href="#">Close a panel</a>                                                                          |

# Glossary

| Term                     | Meaning                                                                                     |
|--------------------------|---------------------------------------------------------------------------------------------|
| <b>Spec file</b>         | A file containing tests, e.g. <code>tests/shop.spec.js</code>                               |
| <b>Step</b>              | One recorded action in a test, shown as one line of the story                               |
| <b>Locator</b>           | How Playwright finds an element, e.g. <code>getByRole('button', { name: 'Sign in' })</code> |
| <b>Assertion / check</b> | A statement that must be true, e.g. <code>expect(total).toHaveText('\$75')</code>           |
| <b>Soft check</b>        | A check that records a failure but lets the test carry on                                   |
| <b>Retry</b>             | Running a failed test again to see whether it passes                                        |
| <b>Flaky</b>             | Failed once, passed on a retry — unreliable rather than broken                              |
| <b>Worker</b>            | One parallel browser running tests                                                          |
| <b>Trace</b>             | Playwright's detailed recording, opened with <code>npx playwright show-trace</code>         |
| <b>Project</b>           | A named browser or device setup, e.g. Desktop Chrome or Mobile Chrome                       |

## What next?

- Run the demo a few more times. The **Results over time** chart and each test's **Stability** strip fill up with history.
- Break something in `demo-app/index.html` — rename the **Sign in** button — and watch how many tests fail, and how Aurora groups them under one cause.
- Add Aurora to a real project with [GUIDE.md](#).



*Tip: the  button in the top right switches between light and dark.*